DBMS MODULE DESIGN DOCUMENT

PoolUp — Users + Ride Creation / Search

PostgreSQL 16 · Node.js / Express · React · Member 1 · 3 October 2026

Module purpose	Let BITS Hyderabad students register, sign in, offer a shared cab ride (with up to three intermediate stops) and find rides that pass through the places they want to go — with every business rule enforced inside the database rather than in application code.
Files in scope	db/00_standins.sql · db/10_users_rides_schema.sql · db/20_logic.sql · db/30_views.sql · db/40_indexes.sql · server/scripts/seed.js · server/src/queries.js · server/src/lab.js
At a glance	**2,728** seeded rows · **47** declarative constraints (22 CHECK, 11 FOREIGN KEY, 9 PRIMARY KEY, 5 UNIQUE) · **6** triggers · **6** stored functions · **2** views · **8** secondary indexes · **11** analytical queries · **12/12** automated tests passing · **12/12** Integrity-Lab scenarios behaving as designed.

1. Requirements Analysis

1.1 Problem

Students travelling from campus to the airport, railway stations and malls often pay for a whole cab alone, or coordinate through scattered WhatsApp groups. PoolUp lets one student offer a ride, and other students find a ride heading their way, so cab fares are shared. This module is the foundation the other modules build on: it owns **who the users are** and **which rides exist**, and answers the question “which rides can I join?”.

1.2 Users

User	What they do
Student (rider / creator)	Registers with a BITS ID, signs in, offers a ride with a time window, optional stops and a cab type, searches for rides, cancels their own ride, edits their profile.
Admin / evaluator	Views the campus dashboard (KPIs, trends, demand by hour, popular routes, hostel league), the SQL library with EXPLAIN ANALYZE, and the Integrity Lab that proves the rules live.

1.3 Functionality

- **Account:** register, log in / out (opaque session token), view and edit profile; public profile exposes only name, hostel and ride counts.
- **Offer a ride:** origin, destination, up to 3 ordered stops, cab type, departure window (≤ 3 h), note. The database computes road distance and the fare estimate; the creator automatically takes the first seat.
- **Search:** find open rides where the searcher’s origin comes *before* their destination along the ride’s route (including via stops), filtered by time window, seats needed and cab type; own rides are excluded; results show seats left and current fare per head.
- **Manage:** “My rides” (created and joined), cancel a ride (creator only) with a status history written by a trigger.
- **Dashboard & SQL library:** ten live analytical queries, each viewable as SQL with its DBMS concepts and plan.

1.4 Assumptions

- A BITS ID follows `YYYY[A|B][digit/letter](PS|TS)NNNNH` (13 chars) and the e-mail is on the campus domain; phones are 10-digit Indian mobile numbers.
- Locations are a fixed set of 16 well-known places (campus gates, airport, stations, bus stops, malls, hubs); coordinates are static reference data.

- Road distance ≈ 1.3 × straight-line (haversine) distance summed along the route; fares are an estimate = base fare + per-km rate × distance, split equally among confirmed riders.
- Joining a ride (the `ride_member` workflow) belongs to Member 2; this module only reads that table for seat counts and auto-inserts the creator.
- A ride's creator, route and cab type are fixed after creation; to change them the creator cancels and re-offers.

1.5 Business Rules and Enforcement

Design principle: **a rule checkable on one row is declarative (CHECK / UNIQUE / FK); a rule that needs other rows or time is a trigger.** The API does not re-implement any of them — it only translates SQLSTATE + constraint name into a readable message.

ID	Rule	Enforced by
U1	BITS ID has the required format	CHECK student_id_format
U2	E-mail must be on the campus domain	CHECK student_email_domain
U3	Phone is a valid 10-digit Indian mobile number	CHECK student_phone_format
U4	Name has ≥ 2 characters	CHECK student_name_len
U5–U7	BITS ID, e-mail and phone are each unique	PRIMARY KEY / UNIQUE ×2
U8	Hostel must exist	FOREIGN KEY
U9	Names / e-mails / IDs are normalised on write	BEFORE trigger student_normalize
R1	Origin and destination differ	CHECK ride_distinct_endpoints
R2	Latest departure is after earliest	CHECK ride_window_valid
R3	Departure window ≤ 3 hours	CHECK ride_window_max_3h
R4	Locations and cab type exist	FOREIGN KEY ×4
R5	A stop appears at most once per ride	UNIQUE ride_stop_unique_location
R6	At most 3 stops per ride	CHECK stop_seq 1..3
R7	Ride starts ≥ 15 minutes from now	BEFORE INSERT trigger ride_validate
R8	Ride created ≤ 60 days ahead	BEFORE INSERT trigger ride_validate
R9	A new ride starts as *open*	BEFORE INSERT trigger ride_validate
R10	A student cannot hold two overlapping active rides	Trigger + row lock (FOR UPDATE) in fn_create_ride
R11	Status flow open⇄full → departed → completed; open/full → cancelled	BEFORE UPDATE trigger ride_status (+ log)
R12	Creator, route and cab type cannot change after creation	BEFORE UPDATE trigger ride_frozen
R13	A stop cannot be the origin or destination	BEFORE INSERT trigger stop_validate
R14	"completed" ⇔ completed_at is set, and not before departure	CHECK ×2
R15	Creator automatically occupies seat 1	AFTER INSERT trigger ride_creator_joins
R16	Every status change is logged	Trigger ride_status → ride_status_log

2. ER / EER Model

The model has eight entities owned by this module plus one dependency (RIDE_MEMBER, Member 2, drawn dashed). A ride is created by exactly one student and travels between two locations; its intermediate stops are a **weak entity** that cannot exist without the ride.

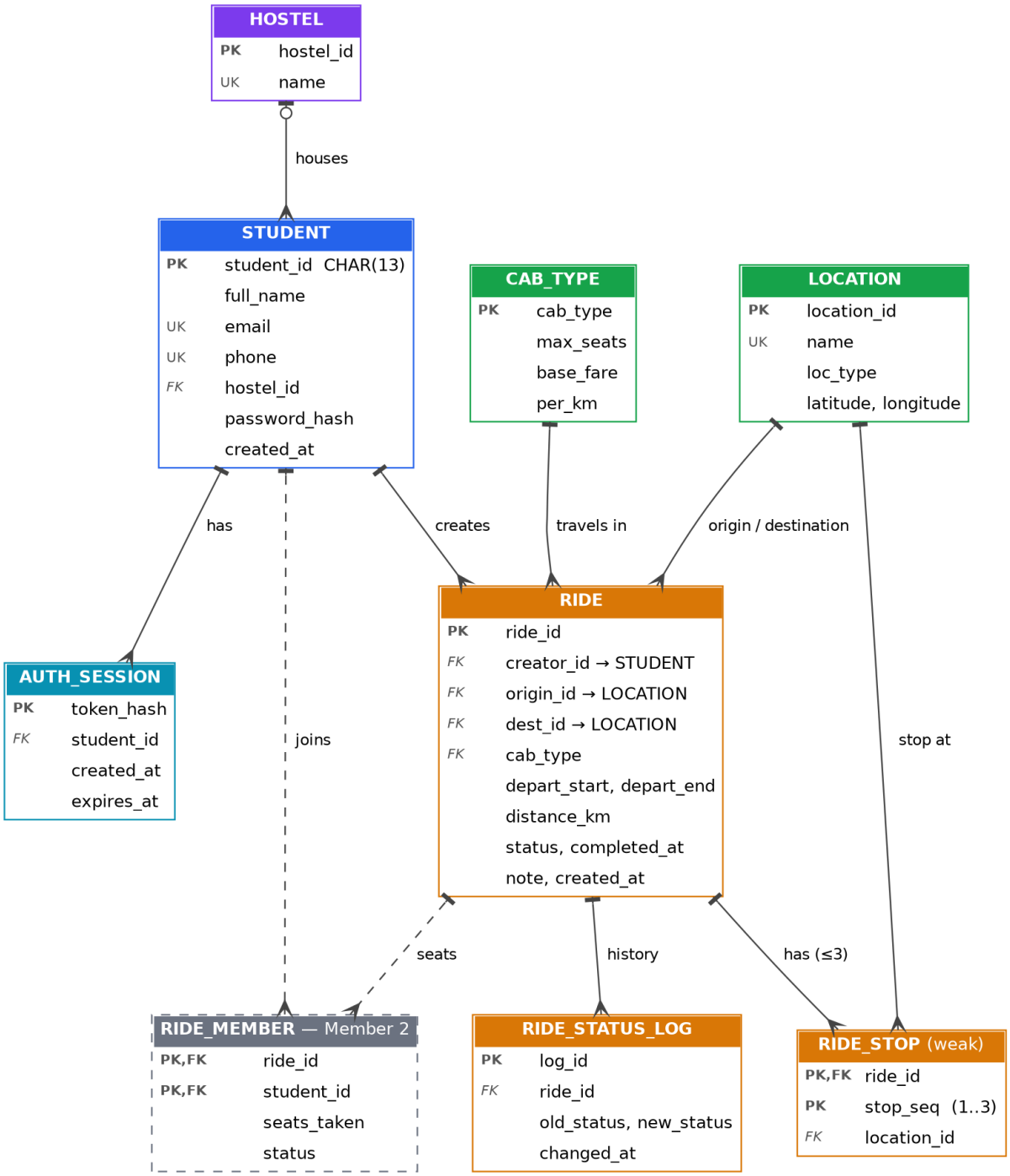


Figure 1 · ER / EER model of the Users + Rides module (crow's-foot notation). Dashed = Member 2's RIDE_MEMBER.

Entity	Key / important attributes
HOSTEL	hostel_id; name UNIQUE
STUDENT	student_id (BITS ID); full_name; email UNIQUE; phone UNIQUE; hostel_id → HOSTEL; password_hash; created_at
AUTH_SESSION	token_hash (SHA-256 of the random token); student_id; created_at; expires_at
LOCATION	location_id; name UNIQUE; loc_type; latitude; longitude
CAB_TYPE	cab_type; max_seats; base_fare; per_km

Entity	Key / important attributes
RIDE	ride_id; creator_id; origin_id; dest_id; cab_type; depart_start/end; distance_km; status; completed_at; note
RIDE_STOP (weak)	ride_id + stop_seq (1..3); location_id
RIDE_STATUS_LOG	log_id; ride_id; old_status; new_status; changed_at
RIDE_MEMBER (Member 2)	ride_id + student_id; seats_taken; status — read for seat counts, auto-filled for the creator

2.1 Modelling decisions

Construct	Decision	Justification
Weak entity	RIDE_STOP is identified by (ride_id, stop_seq) and cascades on delete.	A stop has no meaning outside its ride; the sequence number gives order and the cap of 3.
Two relationships to LOCATION	RIDE has origin_id and dest_id as separate FKs, plus RIDE_STOP.	Same entity plays three roles; a CHECK keeps origin ≠ destination, a trigger keeps stops off both.
Cardinality	STUDENT 1:N RIDE; HOSTEL 1:N STUDENT; CAB_TYPE 1:N RIDE; RIDE 1:N RIDE_STOP.	Total participation on the N side (every ride has a creator, route and cab).
Reference vs. data	Cab fares live in CAB_TYPE, coordinates in LOCATION.	Changing a fare or a location is one row update; no ride is rewritten.
Derived data	Seats left, fare per head, rider count are computed, not stored.	Avoids update anomalies; always consistent with RIDE_MEMBER.
History	RIDE_STATUS_LOG is trigger-maintained and append-only.	Audit trail with no application involvement.

3. Relational Schema

Keys marked **PK**, foreign keys **FK**. All relations are in the `public` schema of one PostgreSQL 16 database.

Relation	Definition
HOSTEL	(**hostel_id**, name UNIQUE)
STUDENT	(**student_id**, full_name, email UNIQUE, phone UNIQUE, hostel_id → HOSTEL, password_hash, created_at)
AUTH_SESSION	(**token_hash**, student_id → STUDENT ON DELETE CASCADE, created_at, expires_at) CHECK expires_at > created_at
LOCATION	(**location_id**, name UNIQUE, loc_type, latitude, longitude) loc_type ∈ {campus, airport, railway, bus, mall, hub}
CAB_TYPE	(**cab_type**, max_seats, base_fare, per_km) max_seats 1..7
RIDE	(**ride_id**, creator_id → STUDENT, origin_id → LOCATION, dest_id → LOCATION, cab_type → CAB_TYPE, depart_start, depart_end, distance_km, note, status, completed_at, created_at)
RIDE_STOP	(**ride_id** → RIDE ON DELETE CASCADE, **stop_seq**, location_id → LOCATION) PK (ride_id, stop_seq); UNIQUE (ride_id, location_id)
RIDE_STATUS_LOG	(**log_id**, ride_id → RIDE ON DELETE CASCADE, old_status, new_status, changed_at)
RIDE_MEMBER*	(**ride_id** → RIDE, **student_id** → STUDENT, seats_taken, status, joined_at) *Member 2; stand-in provided

3.1 Domains and data types

Column	Type	Why
student_id	CHAR(13) + regex CHECK	BITS IDs are fixed-length; the regex rejects typos at the door.
email / phone	VARCHAR(120) / CHAR(10)	Domain-restricted e-mail; phone is an identifier, not a number (no arithmetic, keeps leading digits).
password_hash	TEXT, length ≥ 40	scrypt output “salt:hash”; the DB refuses obviously un-hashed values.
depart_start/end, created_at	TIMESTAMPTZ	Time-zone-safe instants; the UI shows IST, analytics convert with AT TIME ZONE.
latitude / longitude	NUMERIC(8,5)	Exact decimals (~1 m precision), range-checked.
distance_km	NUMERIC(6,1) CHECK > 0	One decimal is enough for fare estimation; zero-length rides are impossible.
status	VARCHAR(10) + CHECK in (…5 values…)	Small closed vocabulary; transitions guarded by a trigger.
fare columns	NUMERIC(7,2) / NUMERIC(6,2)	Money must be exact, never floating point.

4. Normalization

Relation	Candidate keys	Result
STUDENT	student_id, email, phone	Every determinant is a candidate key → **BCNF**. Hostel name is not repeated per student (it would create student → hostel_id → name, a transitive dependency); it lives in HOSTEL.
RIDE	ride_id	All non-key attributes depend on ride_id alone → **3NF/BCNF**. Cab seats and fares are factored into CAB_TYPE (ride → cab_type → max_seats would be transitive); location names/coordinates are in LOCATION.
RIDE_STOP	(ride_id, stop_seq), (ride_id, location_id)	Replaces a multi-valued “stops” attribute (violating 1NF) with one row per stop → **BCNF**.
AUTH_SESSION / LOG	token_hash / log_id	Single-attribute keys, no partial or transitive dependencies → **BCNF**.

Deliberate choice	**distance_km is stored** on RIDE although it could be recomputed from the coordinates: it is a *snapshot at creation* of static reference data and search needs it for every candidate row. Seats left and fare per head are **not** stored — they are derived from RIDE_MEMBER so they can never go stale.

5. Triggers, Functions and Views

Trigger violations raise `ERRCODE 23514` with `CONSTRAINT = rule_…`, the same convention as declarative CHECK failures, so the API reports both uniformly. Seeding loads historical rides under `SET LOCAL poolup.seeding = 'on'`, which bypasses only the time-based trigger rules; every CHECK, FK and UNIQUE still applies.

Object	Kind	Purpose
`fn_create_ride`	Function (plpgsql)	Atomic “create ride”: locks the creator row, computes distance, inserts the ride and each stop. The ride_creator_joins trigger then adds the creator’s seat. All-or-nothing.
`fn_search_rides`	Function (SQL, set-returning)	The search algorithm (section 6.2): UNION path + self-join, seats, window, cab, exclusion; exact-route matches first.
`fn_route_km / fn_haversine_km`	Functions	Great-circle distance × 1.3 summed along origin → stops → destination.
`fn_estimate_fare / fn_seats_left`	Functions	base_fare + per_km × km; max_seats − confirmed seats.
`student_normalize`	BEFORE INSERT/UPDATE	Trims, collapses spaces, lower-cases e-mail, upper-cases ID.
`ride_validate`	BEFORE INSERT	R7–R10: future start, 60-day horizon, starts open, no overlapping ride for the creator.
`ride_creator_joins`	AFTER INSERT	Creator takes seat 1 in RIDE_MEMBER (R15).
`ride_status`	BEFORE UPDATE OF status	State machine (R11) and log row (R16).
`ride_frozen`	BEFORE UPDATE	Creator, route and cab type immutable (R12).
`stop_validate`	BEFORE INSERT	A stop cannot equal origin or destination (R13).
`v_ride_board / v_student_ride_stats`	Views	Seat-aware ride list and per-student created/joined counts used by “My rides” and profile.

5.1 Ride status state machine

From	Allowed next states
open	full · departed · cancelled
full	open · departed · cancelled
departed	completed
completed / cancelled	— (terminal)

5.2 Excerpt — atomic ride creation (db/20_logic.sql)

```
CREATE OR REPLACE FUNCTION fn_create_ride(p_creator text, p_origin int, p_dest int, p_cab text,
    p_start timestamptz, p_end timestamptz, p_stops int[] DEFAULT '{}', p_note text DEFAULT NULL)
RETURNS int LANGUAGE plpgsql AS $$
DECLARE v_id int; v_km numeric; i int;
BEGIN
  PERFORM 1 FROM student WHERE student_id = p_creator FOR UPDATE;   -- serialise same-student creates
  IF NOT FOUND THEN RAISE EXCEPTION 'Unknown student' USING ERRCODE = 'P0002'; END IF;
  IF cardinality(p_stops) > 3 THEN
    RAISE EXCEPTION 'At most 3 stops are allowed'
      USING ERRCODE = '23514', CONSTRAINT = 'ride_stop_stop_seq_check';
  END IF;
  v_km := fn_route_km(p_origin, p_dest, p_stops);
  INSERT INTO ride (creator_id, origin_id, dest_id, cab_type, depart_start, depart_end, distance_km, note)
  VALUES (p_creator, p_origin, p_dest, p_cab, p_start, p_end, v_km, NULLIF(trim(p_note), ''))
  RETURNING ride_id INTO v_id;                  -- triggers: validate (before), creator seat (after)
```

```
  FOR i IN 1 .. cardinality(p_stops) LOOP
    INSERT INTO ride_stop (ride_id, stop_seq, location_id) VALUES (v_id, i, p_stops[i]);
  END LOOP;
  RETURN v_id;
END $$;
```

6. Queries

The module exposes **11 named analytical queries** plus the search function. All are raw parameterised SQL (no ORM) so every concept is visible; each can be opened from the dashboard (SQL button) or the SQL library page with its EXPLAIN ANALYZE plan.

#	Query (server/src/queries.js)	DBMS concepts	Rows	ms
1	**popular_routes** Most popular routes (window function RANK)	GROUP BY, RANK() OVER, JOIN x2, derived table	10	22.5
2	**busiest_hours** Departure demand by hour of day	EXTRACT, GROUP BY, timezone conversion	12	1.3
3	**hostel_league** Hostel league: who creates the most rides?	LEFT JOIN, GROUP BY, HAVING, aggregate	12	2.2
4	**top_creators** Top ride creators (correlated subquery vs average)	correlated subquery, scalar subquery, HAVING	10	1.3
5	**fare_by_cab** Estimated fare per head by cab type and fill level	JOIN, CASE, stored function, aggregate	4	44.7
6	**unfilled_rides** Open rides still looking for riders (anti-join + view)	view, NOT EXISTS, ORDER BY	15	3.5
7	**stop_hotspots** Where do rides make intermediate stops? (weak entity join)	weak entity, JOIN, GROUP BY	10	0.8
8	**monthly_growth** Rides per month with growth (LAG)	date_trunc, LAG() OVER, window function	8	0.9
9	**campus_kpis** Headline KPIs (CTE + conditional aggregation)	CTE, FILTER, scalar subqueries, stored function	1	4.7
10	**cab_mix** Cab type mix and occupancy	GROUP BY, JOIN, derived table, aggregate	4	41.6
11	**status_mix** Ride status breakdown	GROUP BY, window function (share of total)	4	0.5

6.1 Sample results (live from the seeded database)

popular_routes — Most popular routes (window function RANK)

origin	destination	rides	avg_km	popularity_rank
BPHC Main Gate	RGIA Airport (Shamshabad)	81	52.5	1
RGIA Airport (Shamshabad)	BPHC Main Gate	69	52.3	2
BPHC Main Gate	Secunderabad Jn	38	23	3
Secunderabad Jn	BPHC Main Gate	34	33.4	4
BPHC Main Gate	Hitech City	32	34	5

hostel_league — Hostel league: who creates the most rides?

hostel	students	rides_created	rides_per_student
Gautam	13	87	6.69
Meera	17	83	4.88
Ram	13	79	6.08
Malaviya	10	74	7.4
Valmiki	8	61	7.63

status_mix — Ride status breakdown

status	rides	share_pct
completed	554	79.1
open	90	12.9
cancelled	53	7.6
full	3	0.4

cab_mix — Cab type mix and occupancy

cab_type	rides	avg_occupancy_pct
sedan	204	59.2
mini	188	59.2
suv	85	52
auto	77	68

6.2 Search algorithm

Every ride has a **path**: `origin (seq 0) → stop 1…3 → destination (seq 99)`. A search for A → B matches a ride when both A and B appear on its path with **seq(A) < seq(B)**. So “Main Gate → Ameerpet” finds a Main Gate → Airport ride that stops at Ameerpet, while “Ameerpet → Main Gate” does not. Seats must cover the request, the ride’s window must overlap the searcher’s, and the searcher’s own rides are excluded. Exact-route matches are ranked first.

```
WITH path AS (                                  -- one row per point on each open ride's route
  SELECT r.ride_id, 0 AS seq, r.origin_id AS loc FROM ride r WHERE r.status = 'open'
  UNION ALL SELECT s.ride_id, s.stop_seq, s.location_id FROM ride_stop s
  UNION ALL SELECT r.ride_id, 99, r.dest_id FROM ride r WHERE r.status = 'open'),
matched AS (                                    -- self-join: origin comes BEFORE destination
  SELECT DISTINCT a.ride_id
  FROM path a JOIN path b ON b.ride_id = a.ride_id AND b.seq > a.seq
  WHERE (p_origin IS NULL OR a.loc = p_origin) AND (p_dest IS NULL OR b.loc = p_dest))
SELECT ... FROM matched m JOIN ride r USING (ride_id) JOIN cab_type c ... LEFT JOIN occupied o ...
WHERE r.depart_end >= COALESCE(p_from, now()) AND (p_to IS NULL OR r.depart_start <= p_to)
  AND c.max_seats - COALESCE(o.used, 0) >= GREATEST(p_seats, 1)
ORDER BY (r.origin_id = p_origin AND r.dest_id = p_dest) DESC, r.depart_start;
```

On the demo dataset a full two-week search takes about **1.6 ms** execution time (`EXPLAIN ANALYZE`).

7. Indexing and Performance

With only 700 demo rides PostgreSQL correctly prefers sequential scans, so index benefit is **measured on a scratch database with 200,000 rides** (same schema; 56,000 of them open). Each index was added to the same data and the same query re-run (best of 4 runs, `EXPLAIN (ANALYZE, BUFFERS)`).

Query	Without index	With index	Speed-up
Open rides in a 6-hour window (search / board) `ride_open_time_idx — partial: WHERE status = 'open'`	27.558 ms · Gather → Seq Scan · 2804 buffers	0.081 ms · Bitmap Heap Scan → Bitmap Index Scan ride_open_time_idx · 9 buffers	**340×**
Latest 20 rides of one creator (“My rides”) `ride_creator_idx (creator_id, depart_start DESC)`	11.719 ms · Sort → Seq Scan · 2813 buffers	0.084 ms · Index Only Scan student_pkey → Index Scan ride_creator_idx · 29 buffers	**140×**

Index sizes at 200k rides: ride_creator_idx 7960 kB, ride_open_time_idx 1736 kB, ride_pkey 4408 kB. The partial index is the smallest because it only contains rides that can still be joined — finished and cancelled rides never enter it.

7.1 Index catalogue

Index	Definition	Serves
`ride_open_time_idx`	`(depart_start, depart_end) WHERE status = 'open'`	Search + ride board (partial index)
`ride_route_time_idx`	`(origin_id, dest_id, depart_start)`	Route lookups, popular-routes analytics
`ride_creator_idx`	`(creator_id, depart_start DESC)`	“My rides”, creator history, overlap check
`ride_stop_loc_idx`	`(location_id)`	Via-stop matching, stop hotspots
`ride_member_student_idx`	`(student_id, status)`	Rides joined, seat counting
`student_name_prefix_idx`	`(lower(full_name) text_pattern_ops)`	Expression index for name-prefix search
`student_hostel_idx`	`(hostel_id)`	Hostel league join
`session_student_idx`	`(student_id)`	Session cleanup per student

8. Transactions and Concurrency

Concern	Mechanism	Evidence
Create ride is all-or-nothing	fn_create_ride runs in one transaction (ride + stops + creator seat); API wraps it in BEGIN/COMMIT/ROLLBACK.	Test “failed ride leaves no orphan rows”: row count unchanged after a rejected create.
Two simultaneous creates by one student	SELECT … FOR UPDATE on the creator’s STUDENT row serialises them before the overlap check.	Test: 4 parallel requests for the same window → exactly **1** ride created, 3 rejected with rule_no_overlap.
Status integrity	BEFORE UPDATE trigger enforces the legal state graph; nobody (including SQL console) can reopen a cancelled ride.	Integrity Lab scenario “Re-open a cancelled ride” → rejected.
Isolation level	PostgreSQL default READ COMMITTED is sufficient: the only cross-row invariant (no overlap) is protected by an explicit row lock rather than by raising isolation.	Concurrency test above.
Safe demonstrations	Integrity Lab runs every scenario in BEGIN … ROLLBACK.	Test asserts ride / student / stop row counts identical before and after all 12 scenarios.

9. Application Architecture

React (Vite) → Express REST API → PostgreSQL via pg with raw, parameterised SQL. Passwords use scrypt with a per-user salt; sessions are 256-bit random tokens of which only the SHA-256 is stored (7-day expiry).

Endpoint	Auth	Purpose
`POST /api/auth/register · /login · /logout`	public / token	Account lifecycle; register returns a session token
`GET /api/auth/me · PUT /api/auth/me`	token	Own profile with ride counts; edit name / phone / hostel
`GET /api/students?q= · /api/students/:id`	token	Name-prefix search; public profile (no e-mail or phone)
`GET /api/rides/estimate`	token	Distance + fare from fn_route_km / fn_estimate_fare
`GET /api/rides/search`	token	fn_search_rides with all filters
`POST /api/rides`	token	fn_create_ride in a transaction
`GET /api/rides/mine · /api/rides/:id · POST /api/rides/:id/cancel`	token	My rides, ride detail with members and history, creator-only cancel
`GET /api/insights · /:name · /:name/explain`	token	Named analytical queries and their EXPLAIN ANALYZE
`GET /api/lab · POST /api/lab/:id/run`	token	Integrity Lab scenarios (always rolled back)
`GET /api/meta`	public	Hostels, locations, cab types for forms

9.1 Error translation

`server/src/errors.js` maps SQLSTATE to HTTP status (23514 → 422, 23505 → 409, 23503 → 422, 22xxx → 400) and each constraint/trigger name to a rule id and human text, e.g. `rule_no_overlap` → “R10 · You already have a ride in this time window”. The UI shows the mechanism chip (CHECK / UNIQUE / FOREIGN KEY / Trigger) so it is obvious *the database* refused the request.

10. Testing and Verification

`npm test` (Node test runner + supertest) re-creates the database from the SQL files and seed, then exercises the real API against PostgreSQL. **12 / 12 pass**.

#	Test	What it proves
1	Seed volume	≥ 500 rides, ≥ 100 students, stops present
2	Registration	Valid → 201 + normalised name; duplicate → 409; bad e-mail / ID / phone → 422 with the exact constraint
3	Login / me	Wrong password 401; protected route needs a token
4	Create ride	Returns fare, distance, stops; creator holds seat 1 (3 seats left in a 4-seat sedan)
5	Rule rejections	9 invalid rides each rejected by the intended CHECK / FK / UNIQUE / trigger
6	Overlap + atomicity	Overlapping ride rejected; failed create leaves no orphan rows
7	Concurrency	4 parallel creates → exactly one succeeds
8	Search	Exact and via-stop matches; wrong direction does not match; own rides excluded; seat filter works
9	Cancel + state machine	Non-creator gets 403; cancelled → open rejected; route change rejected
10	Insights	All named queries run and return rows
11	Direct SQL	ride_distinct_endpoints also guards inserts that bypass the API
12	Integrity Lab	All 12 scenarios behave as documented and nothing is persisted

10.1 Seed data

Table	Rows	Table	Rows
hostel	12	student	120
location	16	ride	700
cab_type	4	ride_stop	252
ride_status_log	3	ride_member	1621
auth_session	0	**Total**	**2,728**

Generated by a seeded PRNG (identical for every teammate) with dates relative to “now”, so there are always upcoming rides to search. Route popularity is deliberately skewed (the airport dominates) and a few power-users create most rides, giving the analytics something real to show.

11. Integration with the other modules

- **Ratings + Analytics (Member 5)** expects student, hostel, location, cab_type, ride, ride_stop, ride_member with these column names — they match.
- **Login:** replace its `X-Student-Id` header with `requireAuth` from server/src/auth.js and read `actingStudent(req)`.
- **ride_member (Member 2):** keep PK (ride_id, student_id) and columns seats_taken / status; set ride.status to *full* or back to *open* when seats fill or free up — the state-machine trigger allows both.
- **Completing a ride:** UPDATE status to *departed*, then to *completed* with completed_at = now() (both transitions are legal).

12. Application Screenshots

All screenshots were captured from the running application (PostgreSQL 16, Express, React) on 3 October 2026 with the seeded dataset. Ride counts of 701 include the one ride created during the demo.

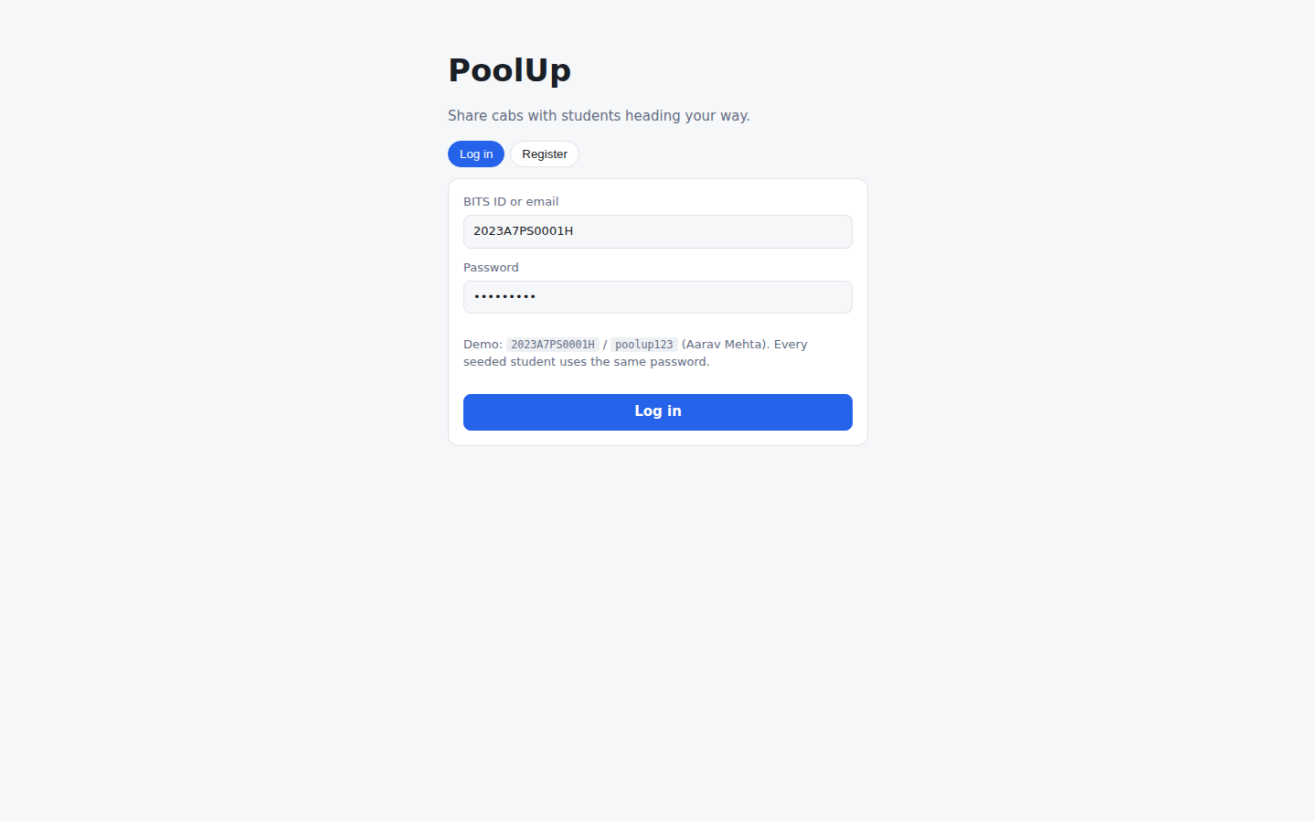


Figure 2 · Sign in / register (demo credentials shown).

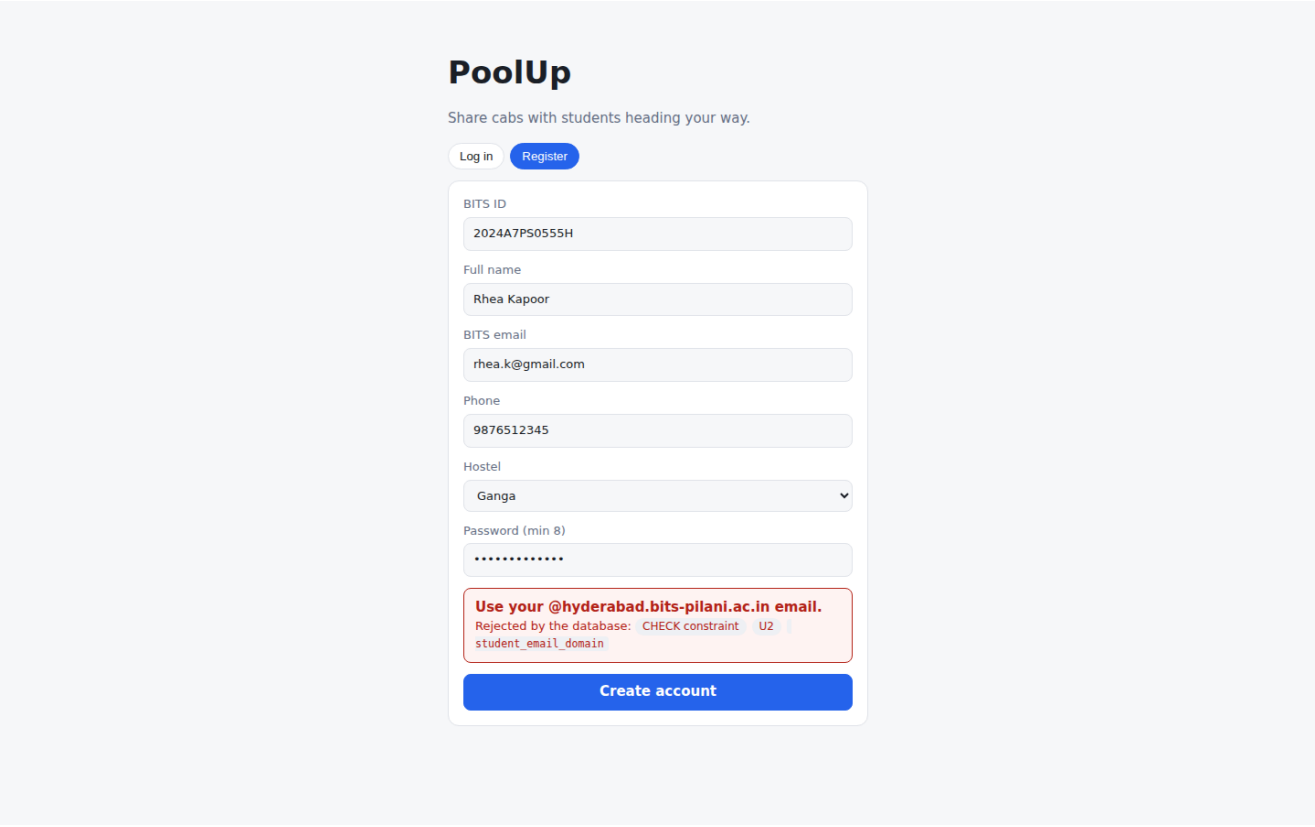


Figure 3 · Registering with a Gmail address — rejected by CHECK student_email_domain (rule U2); the UI shows the mechanism.

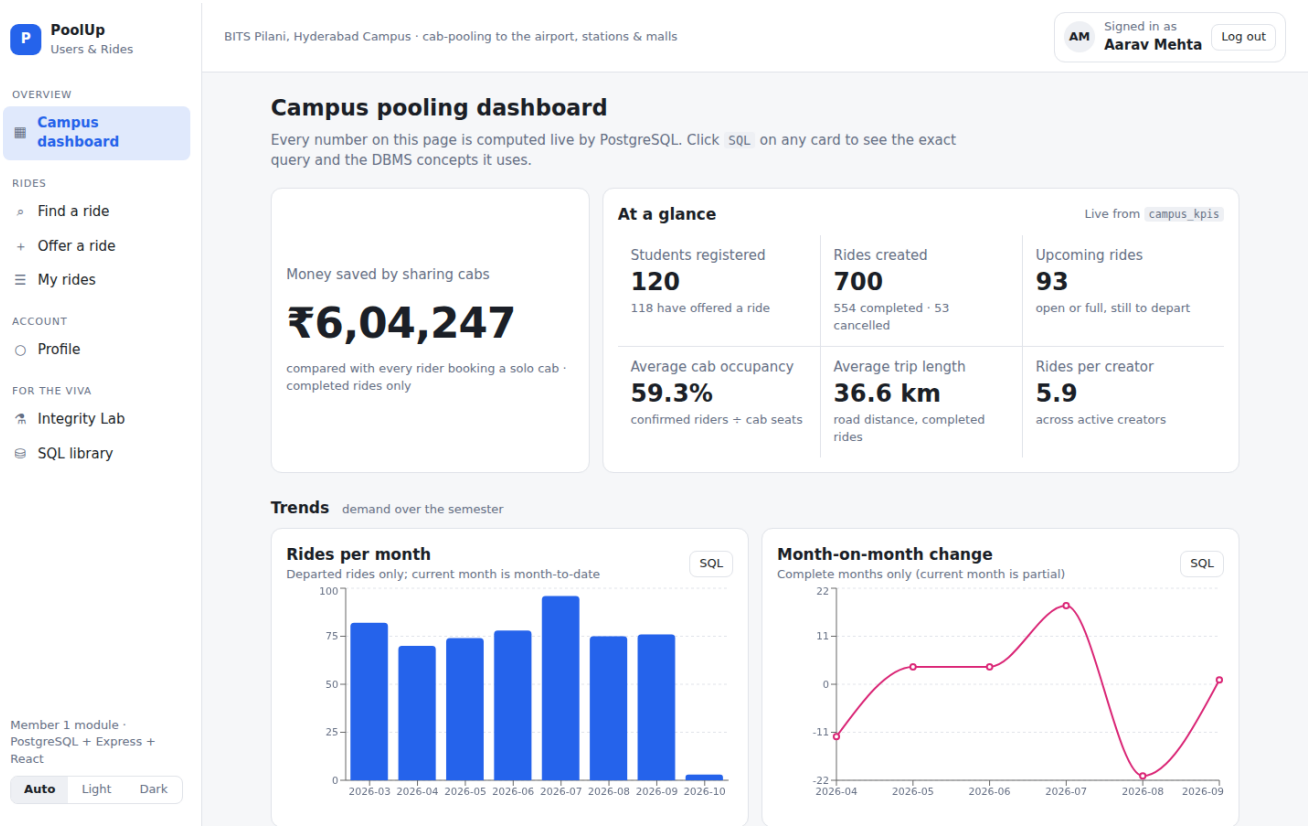


Figure 4 · Campus pooling dashboard: money saved, KPIs and trends, all computed by PostgreSQL.

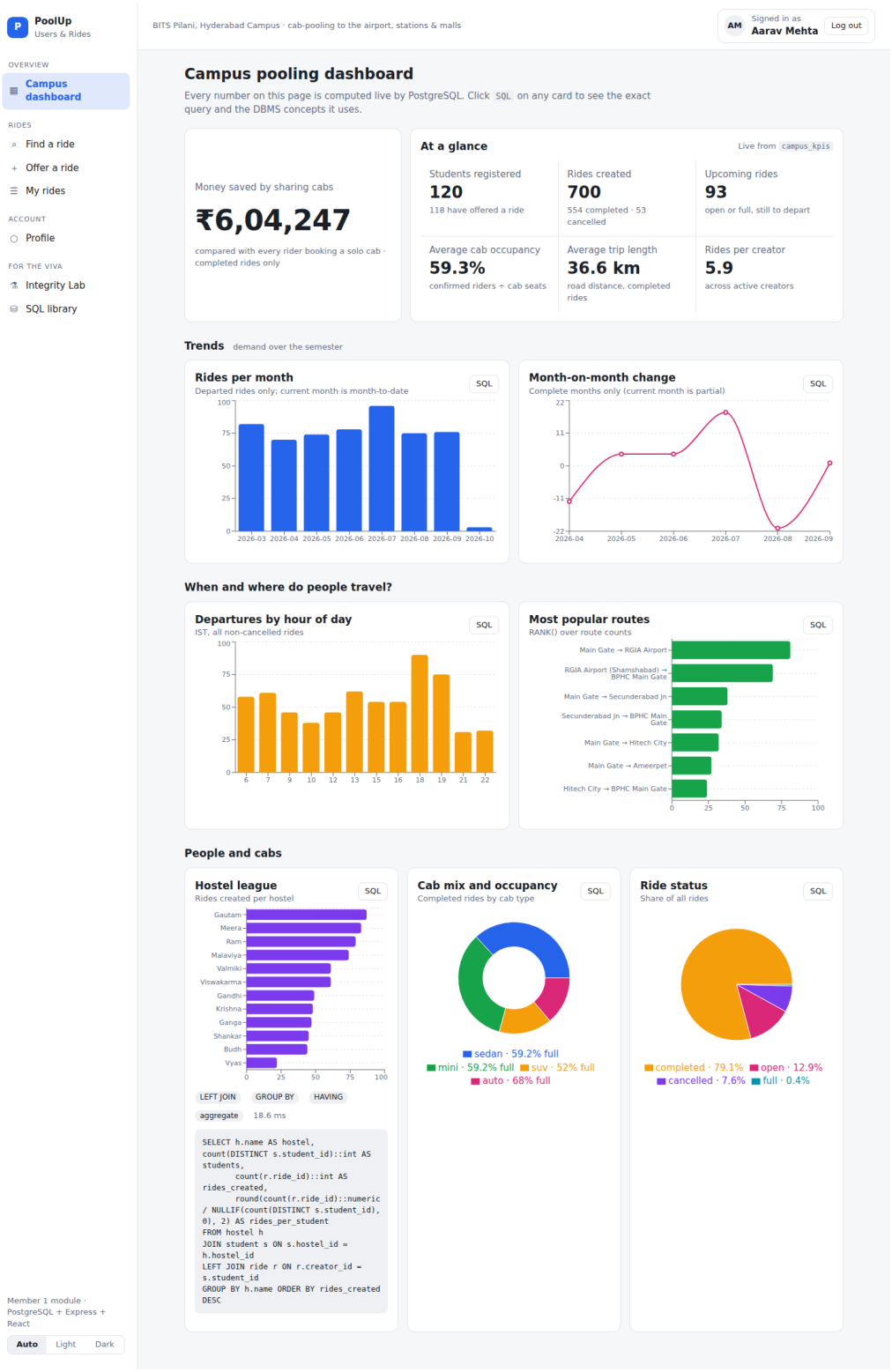


Figure 5 · Full dashboard: demand by hour, popular routes, hostel league, cab mix, ride status; one card opened to show its SQL and concepts.

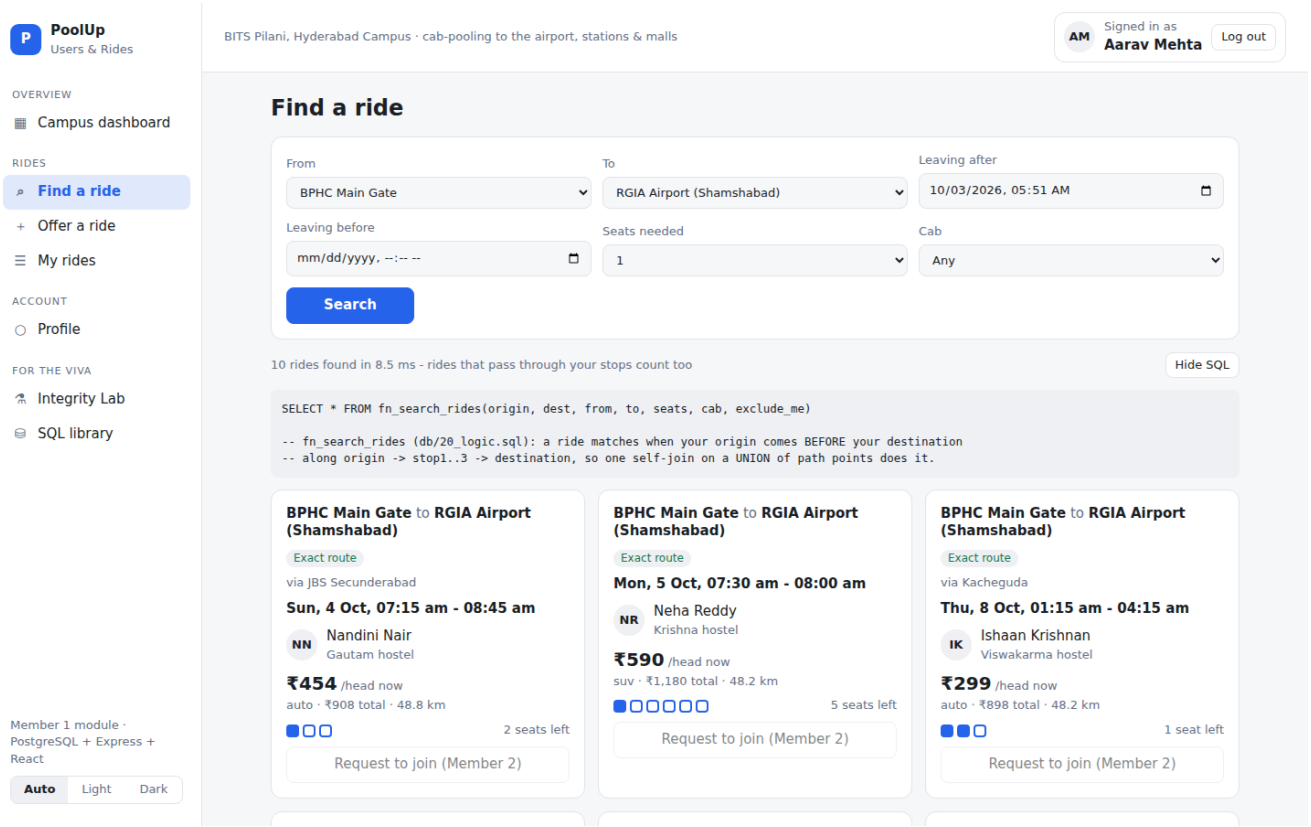


Figure 6 · Ride search from BPHC Main Gate to the airport, with the search SQL panel open; results show seats left and fare per head.

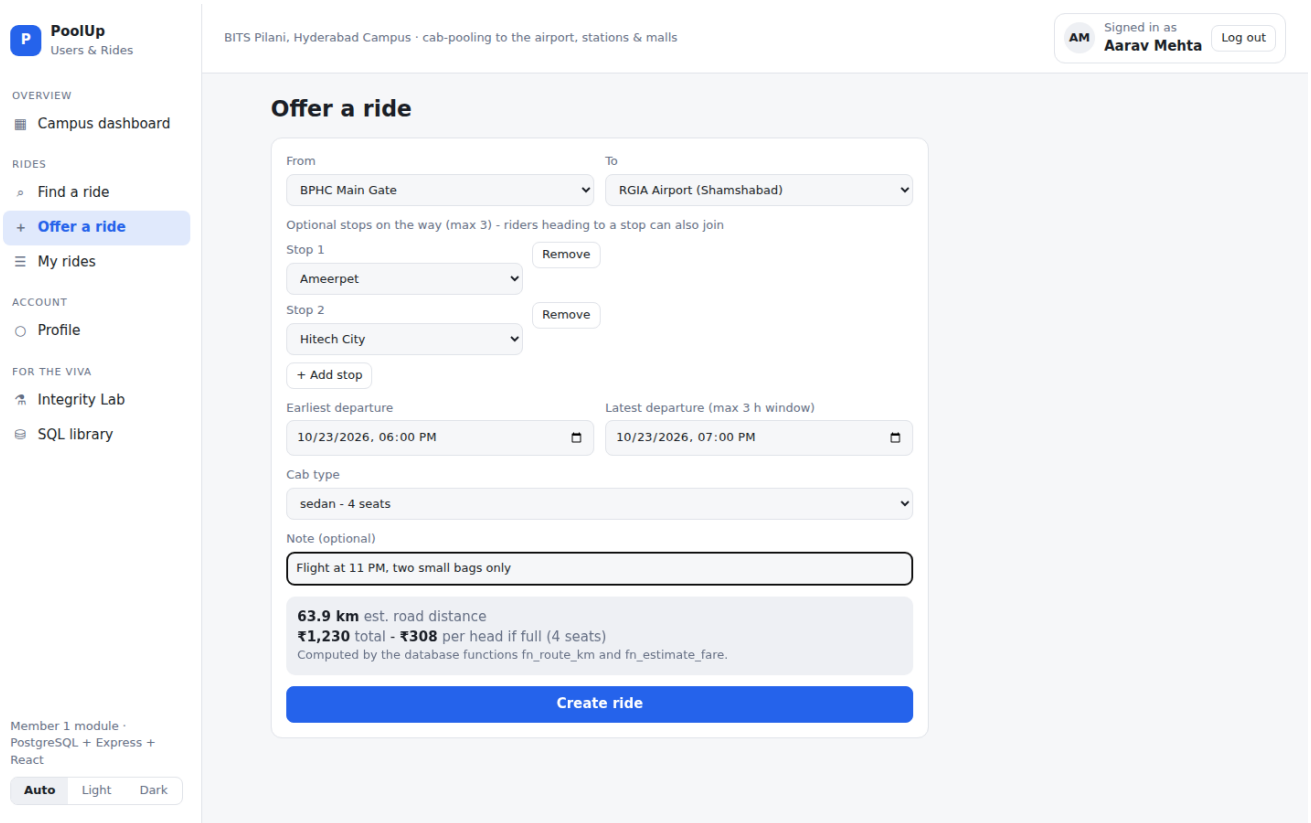


Figure 7 · Offering a ride with two intermediate stops; distance and fare are computed live by fn_route_km / fn_estimate_fare.

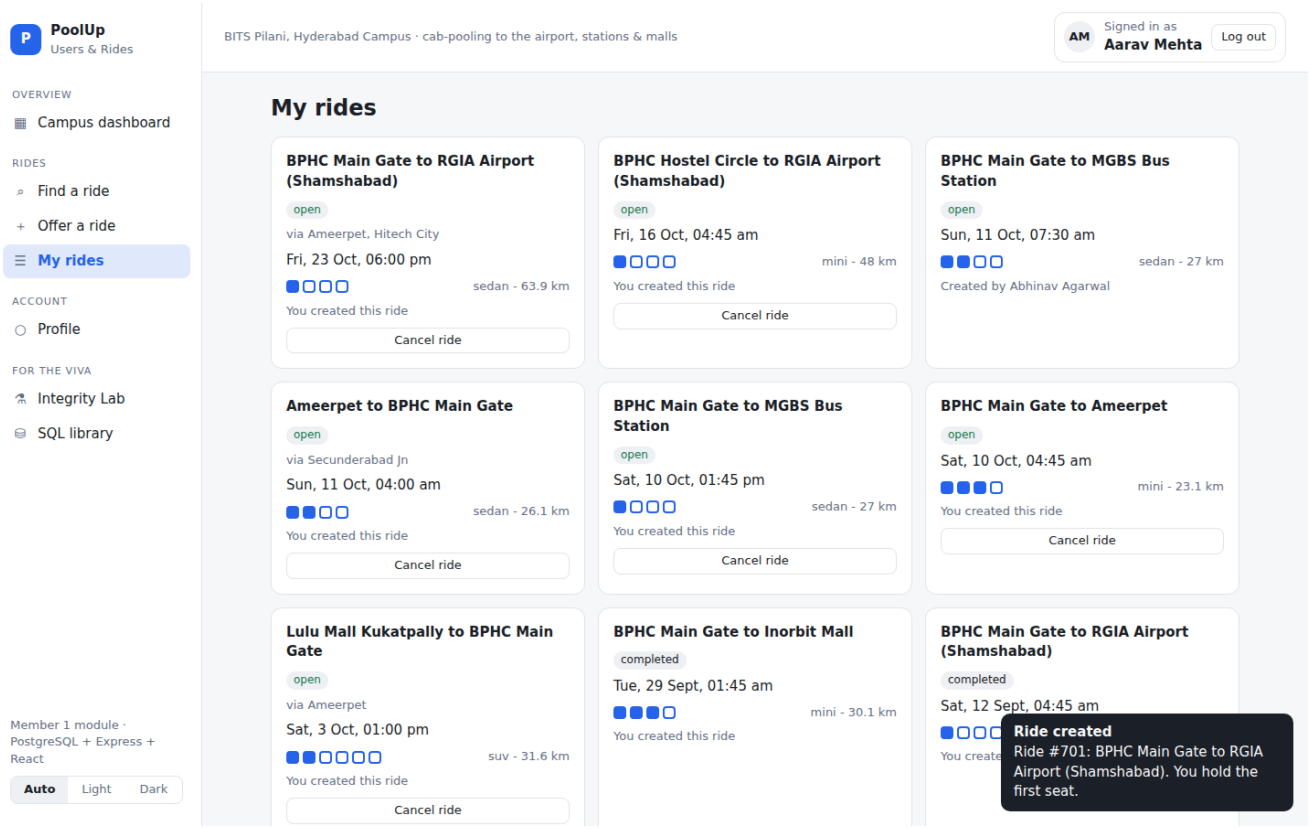


Figure 8 · My rides after creation (the creator automatically holds the first seat).

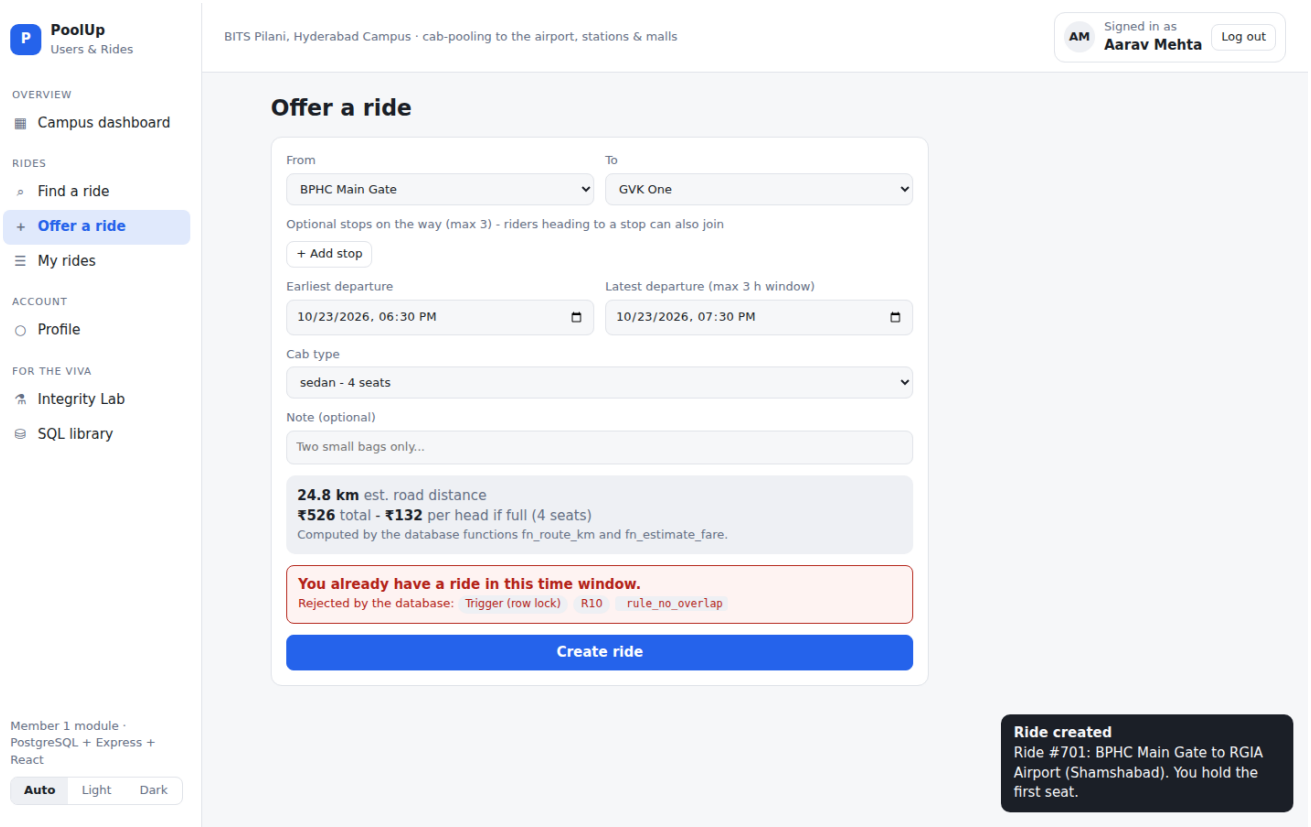


Figure 9 · Creating a second, overlapping ride — rejected by the ride_validate trigger (rule R10).

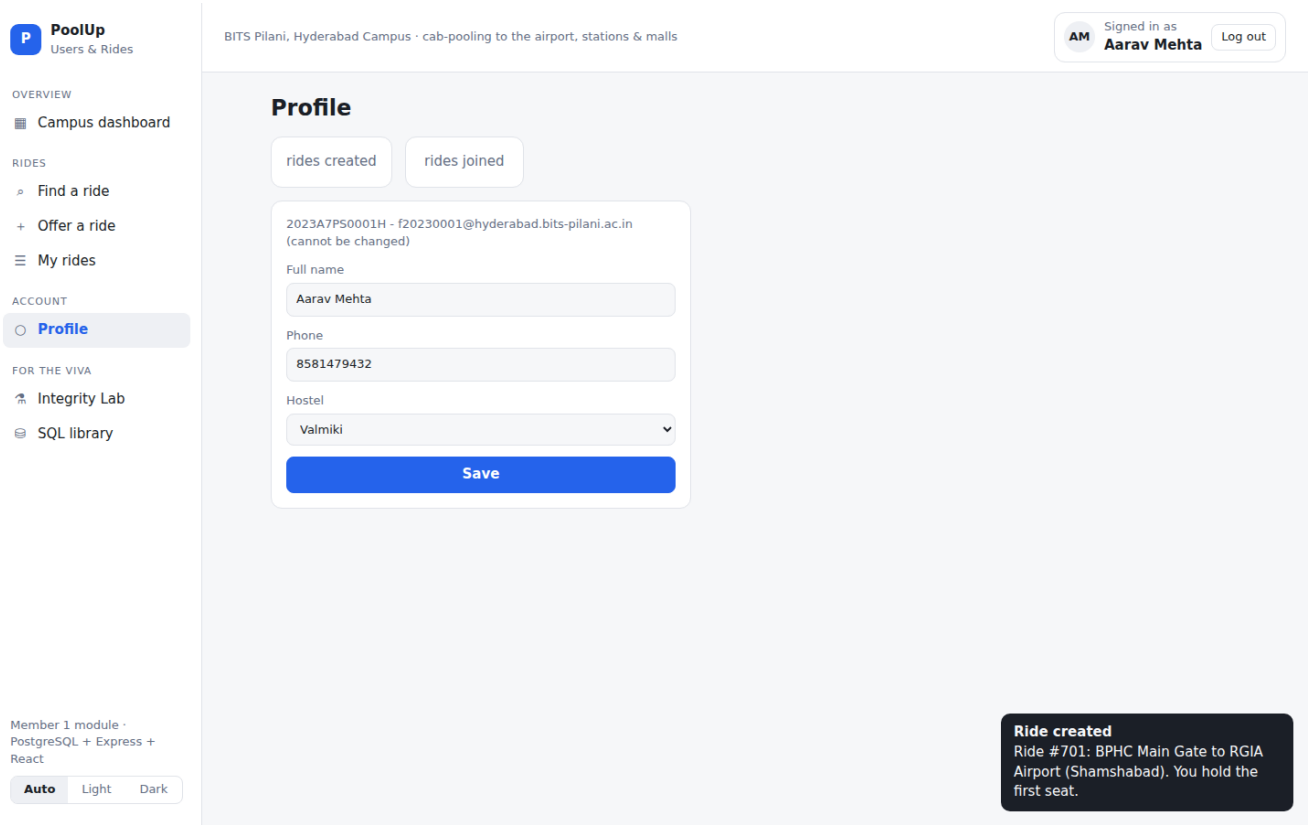


Figure 10 · Profile with created / joined counts from the v_student_ride_stats view.

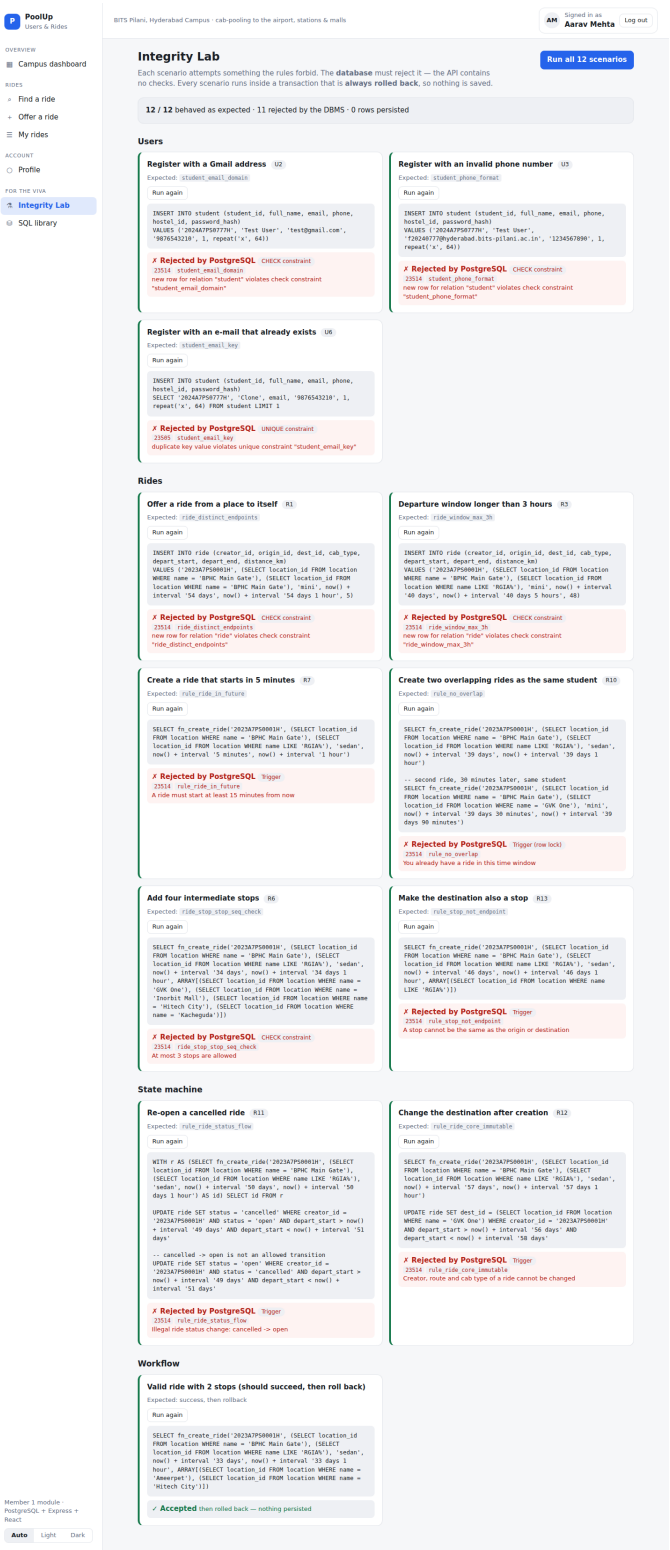


Figure 11 · Integrity Lab: all 12 scenarios run — 11 rejected by the DBMS exactly as expected, 1 valid ride accepted then rolled back; 0 rows persisted.



Figure 12 · SQL library: query, DBMS concepts, result table and EXPLAIN ANALYZE plan.

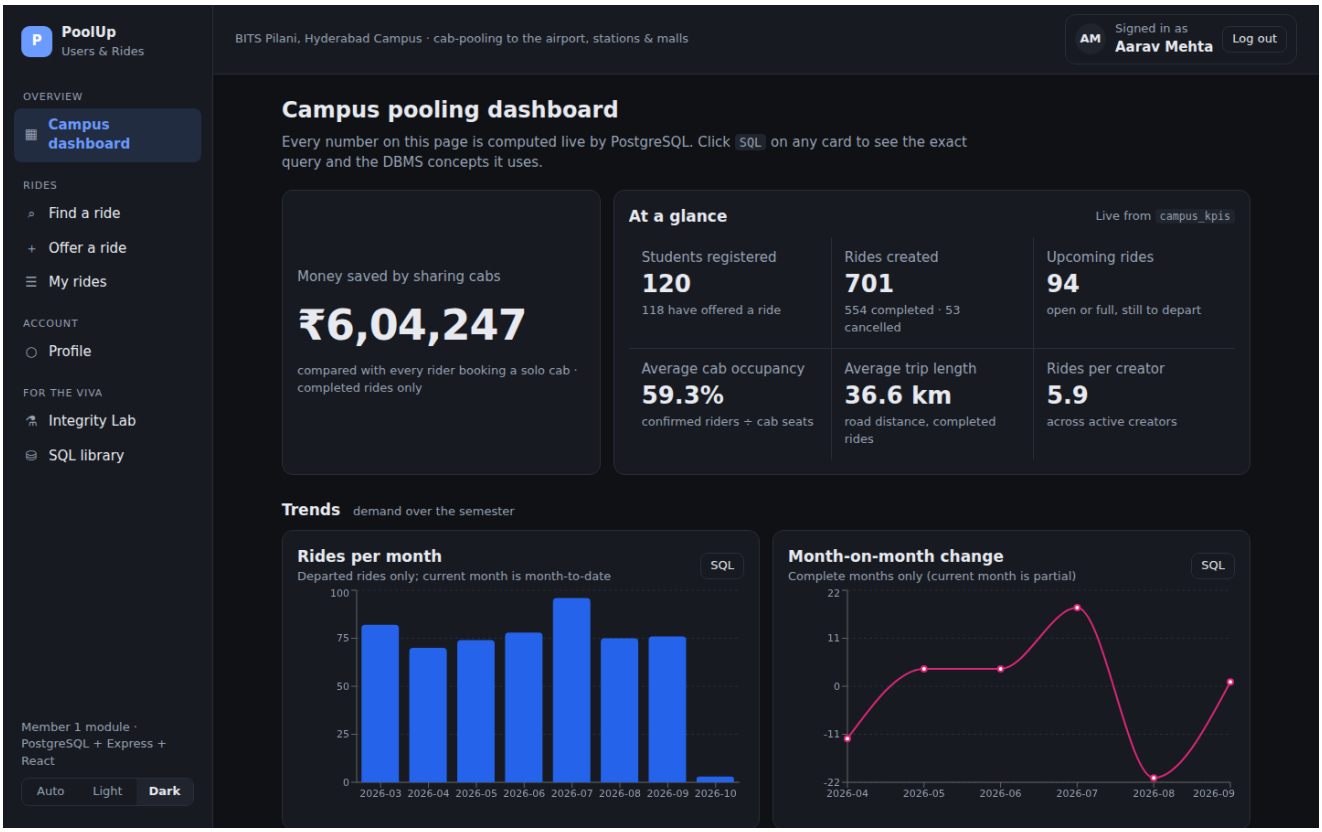


Figure 13 · Dashboard in dark mode (Auto / Light / Dark toggle).